

Universidad del Valle de Guatemala
Facultad de Ingeniería
Departamento de Ciencias de la Computación
Computación Paralela y Distribuida - Sección 20

Proyecto #1

Screensaver de fluidos paralelizado con OpenMP
Simulación de Navier-Stokes (método Stable Fluids) con n-cuerpos

Integrantes

José Antonio Mérida Castejón
Javier Eduardo España Pacheco
Angel Esteban Esquit Hernández

Guatemala, 2 de septiembre de 2026

Índice

1. Introducción	2
2. Antecedentes	3
3. Fundamento Físico - Matemático	4
3.1. Ecuaciones de Navier-Stokes (Incompresibles)	4
3.2. Los 4 operadores, y su versión discreta	4
3.3. Representaciones en Memoria, <code>FluidFields</code>	6
3.4. Paralelización de Gauss-Seidel	6
3.5. N-cuerpos, gravedad y Barnes-Hut	6
4. Descripción del Problema	8
5. Metodología PCAM	9
6. Versión secuencial	10
7. Paralelización iterativa	11
7.1. Paso 1. Barnes-Hut para n-body	11
7.2. Paso 2. Paralelizar los loops independientes	11
7.3. Paso 3. Red-black Gauss-Seidel	11
7.4. Paso 4. Ajuste de <code>schedule</code>	11
7.5. Paso 6. Flags de compilacion	12
8. Mecanismos de sincronización	13
9. Resultados	14
9.1. Tabla maestra	14
9.2. Escalabilidad por hilos	14
9.3. Barrido de <code>schedule/chunk</code>	15
9.4. Flags de build	15
10. Conclusiones	16
11. Recomendaciones	16
A. Anexo 1. Diagrama de flujo del programa	18
B. Anexo 2. Catálogo de funciones	21
C. Anexo 3. Bitácora de pruebas	25

1. Introducción

Este informe documenta el desarrollo del Proyecto #1 del curso de Computación Paralela y Distribuida, un screensaver de fluidos paralelizado con OpenMP. El programa simula un fluido incompresible en 2D usando el método Stable Fluids de Jos Stam, con varias fuentes de tinta de colores que inyectan momentum y color, y que además se mueven entre sí por gravedad mutua (un sistema de n-cuerpos aparte, corriendo sobre la misma malla).

El curso pide diseñar e implementar primero una versión secuencial funcional de un problema con potencial real de paralelización, y luego mejorarla de forma iterativa aplicando el método PCAM y los patrones de descomposición vistos en clase, midiendo speedup y eficiencia en cada paso. La simulación de fluidos se eligió justamente por eso, la malla se puede dividir en regiones independientes de forma natural, pero también tiene un componente genuinamente difícil de paralelizar (el solver de presión y difusión), lo que da margen para mostrar tanto los casos fáciles como una solución algorítmica real a una dependencia de datos.

El proyecto se organizó como una cadena de ramas de git, una por cada paso del roadmap de paralelización, cada una medida por separado antes de integrarse a la rama principal. Este informe resume y explica los resultados, además de cubrir el fundamento matemático del algoritmo y el diseño interno del código.

El resto del informe sigue este orden. La Sección 2 da el contexto de OpenMP y de las técnicas usadas. La Sección 3 deriva las ecuaciones del algoritmo desde Navier-Stokes hasta su forma discreta, la que corre en la malla. Las Secciones de Descripción del problema, Metodología y Versión secuencial cubren el diseño del programa antes de paralelizar. La Sección 7 documenta cada paso de la cadena de paralelización, y la Sección 9 presenta los números de FPS y speedup medidos para cada uno. Los anexos incluyen el diagrama de flujo completo del programa, el catálogo de todas las funciones, y la bitácora de pruebas.

2. Antecedentes

OpenMP es una API para programación paralela de memoria compartida en C/C++/Fortran. Funciona mediante directivas de compilador (`#pragma omp`) que le indican al compilador que partes del código se pueden repartir entre varios hilos. A diferencia de MPI (memoria distribuida), todos los hilos de OpenMP comparten el mismo espacio de memoria, lo cual simplifica la programación pero exige cuidado con condiciones de carrera.

El problema elegido, simulación de fluidos, es un caso clásico de paralelización de memoria compartida. La malla se puede dividir en regiones, y (con el cuidado correcto de dependencias) cada región se puede actualizar en un hilo distinto. El método de Stam (*Stable Fluids*) resuelve las ecuaciones de Navier-Stokes con pasos incondicionalmente estables, lo que permite pasos de tiempo grandes sin que la simulación explote numéricamente [1].

Para el movimiento de las fuentes de tinta se implementó un sistema de n-cuerpos con gravedad mutua, acelerado con un quadtree Barnes-Hut [2], que reduce el costo de $O(n^2)$ a $O(n \log n)$.

3. Fundamento Físico - Matemático

Esta sección define los campos y ecuaciones detrás del algoritmo, desde la ecuación continua hasta la versión discreta que corre en el código. Las versiones paralelas resuelven exactamente las mismas ecuaciones; solo cambia cómo se reparte el trabajo entre hilos.

3.1. Ecuaciones de Navier-Stokes (Incompresibles)

El fluido se describe con dos campos sobre el plano. El campo de velocidad es $\mathbf{u}(x, y, t) = (u_x, u_y)$, y el campo de presión es $p(x, y, t)$. Ambos son funciones continuas del espacio y del tiempo. Para que el fluido sea incompresible, ambos campos deben satisfacer las ecuaciones de Navier-Stokes.

$$\frac{\partial \mathbf{u}}{\partial t} = -(\mathbf{u} \cdot \nabla) \mathbf{u} + \nu \nabla^2 \mathbf{u} + \mathbf{f} \quad (1)$$

$$\nabla \cdot \mathbf{u} = 0 \quad (2)$$

La Ecuación 1 es la conservación de momentum. Dice que $\mathbf{u}$ cambia por 3 efectos independientes que se suman. El primero es la autoadvección, $-(\mathbf{u} \cdot \nabla) \mathbf{u}$, el fluido arrastrándose a sí mismo. El segundo es la difusión, $\nu \nabla^2 \mathbf{u}$, la viscosidad ν suavizando el campo. El tercero son las fuerzas externas, $\mathbf{f}$, lo que las fuentes inyectan directamente. La Ecuación 2 es la restricción de incompresibilidad. La divergencia $\nabla \cdot \mathbf{u}$ mide cuánto más sale de una región que lo que entra; forzarla a cero en todo punto es lo que le impide al fluido comprimirse o expandirse.

Resolver ambas ecuaciones a la vez, de forma exacta, es costoso y numéricamente inestable a pasos de tiempo grandes. Stam [1] las resuelve con *operator splitting*. En vez de tratar los 3 efectos de la Ecuación 1 de forma simultánea, se aplica cada uno por separado, en su propio sub-paso, y al final se corrige la incompresibilidad de la Ecuación 2 aparte. Esa secuencia de sub-pasos es exactamente lo que implementa `velocity_step` en `solver.c`.

El mismo $\mathbf{u}$ de las ecuaciones tiene un tercer uso en el programa. Cada canal de color de la tinta (rojo, verde, azul) es tratado como un campo escalar $d(x, y, t)$ que no empuja al fluido, solo se deja arrastrar por él. d obedece su propia ecuación de transporte pasivo,

$$\frac{\partial d}{\partial t} = -(\mathbf{u} \cdot \nabla) d + \kappa \nabla^2 d,$$

con κ el coeficiente de difusión de tinta (flag `-d`). Es la Ecuación 1 sin el término de fuerza y sin la parte vectorial, y por eso se resuelve con los mismos 2 operadores que la velocidad (difusión y advección, ver abajo) en vez de necesitar código aparte: es lo que hace `ink_step`, una vez por canal.

3.2. Los 4 operadores, y su versión discreta

La malla es una grilla de $(n + 2) \times (n + 2)$ celdas. Las $n \times n$ celdas interiores contienen la simulación real; el anillo de 1 celda alrededor es una frontera fantasma, usada para aplicar las condiciones de borde sin necesitar casos especiales dentro del loop principal (la macro `IX(i, j)` en `common.h` mapea (i, j) al índice plano del arreglo). Cada operador de abajo actúa sobre esta grilla discreta, y todos comparten la misma firma de entrada y salida, un campo antes del paso ($\mathbf{u}^0$ o d^0) y el mismo campo después.

1. Fuerza (`add_source`). Aplica el término $\mathbf{f}$ de la Ecuación 1 directamente sobre la malla,

$$\mathbf{u}_{i,j} \leftarrow \mathbf{u}_{i,j} + \Delta t \mathbf{f}_{i,j},$$

una celda a la vez. Cada celda solo depende de sí misma, así que es trivial de paralelizar. *En código*, es un loop plano de `total_cells` iteraciones con un solo `#pragma omp parallel for schedule(static)` encima, sin variables privadas ni ningún cuidado extra.

2. Difusión (diffuse). El laplaciano $\nabla^2 \mathbf{u}$ se aproxima con diferencias finitas centradas, la diferencia entre una celda y el promedio de sus 4 vecinos. Evaluar esa aproximación en el estado *nuevo* (en vez del viejo) es lo que hace que el método sea incondicionalmente estable, pero convierte cada celda en una incógnita que depende de sus 4 vecinas, también incógnitas. Para toda la malla a la vez, eso es un sistema lineal disperso $A\mathbf{u} = \mathbf{u}^0$ de $N = n^2$ ecuaciones (una por celda), donde A solo tiene 5 entradas no cero por fila. Celda por celda, cada fila de ese sistema se ve así.

$$(1 + 4a) u_{i,j} - a(u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1}) = u_{i,j}^0, \quad a = \nu \Delta t n^2.$$

Un sistema de dimensiones $N \times N$ es demasiado grande para invertir A directamente a cada frame, así que se aproxima de forma iterativa con **Gauss-Seidel**. Cada iteración recalcula toda celda con

$$u_{i,j} \leftarrow \frac{u_{i,j}^0 + a(u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1})}{1 + 4a},$$

usando los valores de vecinos más recientes disponibles (de ahí el nombre, cada pasada mejora la aproximación anterior). El código corre `GAUSS_SEIDEL_ITERS = 20` iteraciones de esto por llamada, que en la práctica converge lo suficiente para verse estable en el rango de `-n` usado. La proyección (operador 4, abajo) resuelve la misma forma de sistema, cambiando $\mathbf{u}$ por p . *En código*, esta fórmula es exactamente lo que calcula `relax_color` en `solver.c`, una vez por color del recorrido red-black (Sección 7.3 explica por qué hace falta ese recorrido en vez de un loop plano); las 20 iteraciones completas corren dentro de una sola región `#pragma omp parallel`, con `#pragma omp single` aplicando la condición de frontera entre colores.

3. Advección (advect). El fluido transporta tanto a $\mathbf{u}$ como a d a lo largo de sí mismo. Para no ser inestable a pasos de tiempo grandes, cada celda destino “mira hacia atrás” en vez de empujar su valor hacia adelante. En vez de preguntar “¿a dónde va mi valor?”, pregunta “¿de dónde vino el valor que ahora tengo?”. Para la celda (i, j) , ese origen es

$$(x_0, y_0) = (i, j) - \Delta t n \mathbf{u}_{i,j}.$$

Como (x_0, y_0) casi nunca cae en una celda entera, el valor final se obtiene por **interpolación bilineal** entre las 4 celdas reales alrededor de ese punto fraccional (el Anexo 1, bloque 4, tiene el detalle línea por línea de esa interpolación). *En código*, cada celda destino solo lee `field_prev`, nunca lo escribe, así que el loop es paralelizable sin ningún cuidado extra, con `#pragma omp parallel for collapse(2)` fundiendo los 2 loops anidados (i, j) en un solo rango repartible y `private(...)` listando las variables escalares temporales (i_0, i_1, j_0, j_1) y los 4 pesos de interpolación) para que cada hilo tenga su propia copia.

4. Proyección de Hodge (project). Los 3 operadores anteriores dejan a $\mathbf{u}$ con algo de divergencia, violando la Ecuación 2. La descomposición de Hodge dice que cualquier campo $\mathbf{w}$ se puede separar en una parte libre de divergencia más el gradiente de un escalar, $\mathbf{w} = \mathbf{u} + \nabla p$, con $\nabla \cdot \mathbf{u} = 0$. Recuperar ese $\mathbf{u}$ toma 3 pasos. Primero se mide la divergencia de $\mathbf{w}$, luego se resuelve la ecuación de Poisson $\nabla^2 p = \nabla \cdot \mathbf{w}$ (mismo solver disperso de la difusión, ahora para p), y finalmente se resta su gradiente, $\mathbf{u} = \mathbf{w} - \nabla p$. El código llama a este operador dos veces por frame (Anexo 1, bloque 3) porque

tanto difundir como autoadvectar reintroducen algo de divergencia cada uno. *En código*, medir la divergencia y restar el gradiente son 2 loops separados, cada uno con su propio `#pragma omp parallel for collapse(2)`; el paso de en medio (resolver Poisson para p) reusa `solve_linear`, la misma función de Gauss-Seidel red-black del operador 2.

3.3. Representaciones en Memoria, FluidFields

Cada campo de arriba vive en memoria como uno o más arreglos de `fields.c/fields.h`, del tamaño de la malla completa. La Tabla 1 conecta cada símbolo con su nombre real en el código.

Símbolo	Campo(s) de FluidFields	Qué guarda
$\mathbf{u} = (u_x, u_y)$	<code>vel_x, vel_y</code>	Velocidad actual, las 2 componentes por separado.
$\mathbf{u}^0$	<code>vel_x_p, vel_y_p</code>	Velocidad del paso anterior; también acumula la fuerza $\mathbf{f}$ antes de cada <code>velocity_step</code> .
p	<code>pressure</code>	Presión, resuelta por <code>project</code> y descartada cada frame.
$\nabla \cdot \mathbf{u}$	<code>divergence</code>	Divergencia medida al inicio de <code>project</code> , lado derecho de la ecuación de Poisson.
d (uno por canal)	<code>ink_r, ink_g, ink_b</code>	Densidad de tinta actual, un campo escalar independiente por color.
d^0 (uno por canal)	<code>ink_r_p, ink_g_p, ink_b_p</code>	Densidad de tinta del paso anterior, y acumulador de lo que inyecta <code>inject_sources</code> .

Cuadro 1: De los símbolos matemáticos a los campos reales de FluidFields.

El patrón “actual” + “p” (anterior) se repite en los 3 grupos de campos porque `diffuse` y `advect` necesitan leer el estado viejo completo mientras escriben el nuevo (Sección de Anexo 2 tiene la firma completa de cada función). Después de cada operador, `swap_fields` intercambia los 2 punteros en vez de copiar el arreglo entero, así que “actual” y “anterior” cambian de lugar sin mover datos.

3.4. Paralelización de Gauss-Seidel

La fórmula de Gauss-Seidel de arriba actualiza $u_{i,j}$ usando $u_{i-1,j}$ y $u_{i,j-1}$, que ya se actualizaron en esa misma pasada (a diferencia de Jacobi, que siempre lee el estado completo de la pasada anterior). Leer valores ya actualizados es lo que hace converger a Gauss-Seidel más rápido que Jacobi, pero también crea una dependencia secuencial estricta entre celdas consecutivas. La celda (i, j) no puede calcularse sin que $(i-1, j)$ ya haya terminado. Repartir ese loop entre hilos tal cual produciría condiciones de carrera. La Sección 7.3 explica la solución que se implementó (un recorrido red-black) para paralelizarlo sin perder la velocidad de convergencia de Gauss-Seidel.

3.5. N-cuerpos, gravedad y Barnes-Hut

Cada fuente de tinta es también un cuerpo, con posición propia (x_i, y_i) , velocidad propia $(v_{x,i}, v_{y,i})$ y masa m_i , independientes de la malla de fluido (viven en la struct `InkSource`, no en `FluidFields`). La fuerza gravitacional entre dos cuerpos i, j es

$$\mathbf{F}_{i \rightarrow j} = G \frac{m_i m_j}{d^2 + \epsilon^2} \hat{\mathbf{d}}, \quad d = |\mathbf{r}_j - \mathbf{r}_i|,$$

con $\hat{\mathbf{d}}$ el vector unitario de i hacia j . El término de *softening* ϵ (NBODY_SOFTENING, un valor fijo en celdas) evita que la fuerza se dispare a infinito cuando $d \rightarrow 0$, algo que rompería la integración numérica cada vez que dos fuentes casi se tocan. Cada frame, `update_nbody_sources` integra esta fuerza con Euler semi-implícito, actualizando primero la velocidad y luego la posición con esa velocidad ya nueva,

$$\mathbf{v}_i \leftarrow \mathbf{v}_i + \sum_{j \neq i} \mathbf{F}_{i \rightarrow j}, \quad \mathbf{r}_i \leftarrow \mathbf{r}_i + \mathbf{v}_i,$$

con $\mathbf{v}_i$ limitada a NBODY_VEL_MAX para que un encuentro cercano no dispare una fuente fuera de pantalla, y con un rebote amortiguado (factor NBODY_RESTITUTION, menor a 1) al llegar a los bordes de la zona válida de inyección. Sumar $\mathbf{F}_{i \rightarrow j}$ para cada par de cuerpos cuesta $O(f^2)$ evaluaciones por frame.

Barnes-Hut [2] baja ese costo a $O(f \log f)$ reemplazando la suma directa por un recorrido de árbol. El espacio se agrupa en un quadtree, una estructura donde cada nodo cubre una región cuadrada y tiene hasta 4 hijos, uno por cuadrante. La inserción (`bh_insert`) es recursiva, cada nodo hoja vacío recibe directamente el primer cuerpo que le toca, y al llegar un segundo cuerpo a una hoja ya ocupada, esa hoja se subdivide en 4 y ambos cuerpos bajan a sus cuadrantes correspondientes. Un guardia de profundidad (BH_MAX_DEPTH) evita recursión infinita cuando 2 fuentes caen casi en el mismo punto, tratándolas como una sola masa combinada a esa profundidad. Cada nodo también acumula la masa total y el centro de masa de todo lo que contiene, actualizándolos según sube la recursión.

Calcular la fuerza sobre un cuerpo (`bh_accumulate`) recorre el árbol desde la raíz. Si el nodo actual es una hoja, o si está lo bastante lejos en relación a su propio tamaño, el criterio

$$\frac{2s}{d} < \theta, \quad \theta = \text{BH_THETA} = 0,5,$$

con s el ancho del nodo y d la distancia hacia su centro de masa, el algoritmo trata todo el nodo como un único cuerpo puntual ubicado en ese centro de masa, en vez de bajar a visitar cada uno de sus hijos por separado. θ más chico visita más nodos y se acerca más al resultado exacto de $O(f^2)$; θ más grande aproxima más agresivo y es más rápido. El Anexo 1, bloque 1, tiene el resto del detalle de implementación línea por línea.

4. Descripción del Problema

El programa dibuja un lienzo de al menos 640x480 px donde varias fuentes de tinta de colores inyectan color y momentum en un campo de velocidad. El campo de velocidad se resuelve cada frame (difusión, advección, proyección de Hodge) y arrastra la tinta, generando remolinos. Las fuentes mismas se mueven por atracción gravitacional mutua y rebotan en los bordes de la malla.

El programa acepta varios parámetros por línea de comandos, entre ellos la resolución de la malla (**-n**), la cantidad de fuentes (**-f**), el tamaño de ventana (**-W/-H**), la semilla aleatoria (**-s**), la viscosidad (**-v**), la difusión (**-d**) y la pantalla completa (**-p**). Los valores por defecto y rangos válidos están en la Tabla 2.

Flag	Rango	Default
-n	16 – 1024	256
-f	1 – 256	6
-W	min. 640	1920
-H	min. 480	1080
-v	0.0 – 1.0	0.0
-d	0.0 – 1.0	0.0001

Cuadro 2: Parámetros de línea de comandos.

5. Metodología PCAM

Se aplicó el método PCAM (Particionamiento, Comunicación, Aglomeración, Mapeo) para decidir qué y cómo paralelizar.

Particionamiento La malla de la simulación se piensa como un conjunto de celdas independientes. Cada operación por celda (inyección de fuente, disipación, advección, proyección, render) es, en principio, una tarea separada.

Comunicación La mayoría de las operaciones solo leen el estado del frame anterior y escriben el nuevo, sin comunicación entre celdas dentro del mismo paso. La excepción es el solver de presión/difusión (Gauss-Seidel), que en su forma original lee valores ya actualizados de celdas vecinas en la misma iteración, creando una dependencia estricta que impide paralelizar directamente.

Aglomeración En vez de repartir celda por celda (overhead de sincronización altísimo), se agrupan filas en bloques (*chunks*) que cada hilo procesa de corrido.

Mapeo Los bloques se reparten entre los hilos disponibles via `#pragma omp parallel for`, con la política de reparto (`schedule`) elegida según el tamaño de la malla (ver Sección 7.4).

6. Versión secuencial

La versión secuencial (rama `sequential`) sirve de línea base fija para medir speedup. Usa Gauss-Seidel fila por fila para el solver de presión/difusión y fuerza bruta $O(f^2)$ para la gravedad del n-body (sin quadtree, cada fuente contra cada otra fuente). El resto de la estructura del programa (parseo de argumentos, asignación de memoria, bucle principal, limpieza al salir) es idéntica a las versiones paralelas. Los pasos del roadmap solo tocan el algoritmo del solver y el n-body, no la estructura general.

Antes de arrancar cualquier paralelización, esta versión se probó. Compila limpio con `-Wall -Wextra -O2 -std=c11`, corre sin crashear para el rango completo de `-n` y `-f`, y `valgrind --leak-check=full` reporta 0 bytes perdidos y 0 errores (la limpieza al salir libera los 12 campos de la simulación, las fuentes y las estrellas de fondo, ver `cleanup:` en `main.c`). También se probó el parseo de argumentos con entradas invalidas (no numericas, fuera de rango, valores faltantes, flags desconocidos), ver Anexo 2 para el detalle de `read_int/read_float`.

7. Paralelización iterativa

Siguiendo el espíritu de OpenMP como herramienta iterativa, la paralelización se hizo en pasos, cada uno en su propia rama, midiendo la misma batería de pruebas después de cada uno.

7.1. Paso 1. Barnes-Hut para n-body

Se reemplazó el cálculo de gravedad de fuerza bruta ($O(f^2)$) por un quadtree Barnes-Hut ($O(f \log f)$). Este paso es puramente algorítmico, sin OpenMP todavía. A las resoluciones de malla probadas, el solver de fluidos domina tanto el costo que el cambio no se nota en el FPS total (el n-body es una fracción pequeña del trabajo por frame).

7.2. Paso 2. Paralelizar los loops independientes

Se agregó `#pragma omp parallel for` a los sitios donde cada celda de salida es independiente, entre ellos la inyección de fuentes, la disipación de tinta, la advección, la proyección de Hodge y el render. El solver de presión/difusión seguía secuencial. Con 20 hilos, este paso ya da un salto importante de FPS (Tabla 3), pero el techo sigue siendo el solver.

7.3. Paso 3. Red-black Gauss-Seidel

El cambio más grande de toda la cadena. El solver de Gauss-Seidel original lee y escribe la misma malla en el mismo barrido, celda por celda en orden, lo que crea una dependencia secuencial estricta. La solución es un recorrido *red-black*. La malla se divide en un tablero de ajedrez por paridad $(i + j) \bmod 2$. Los 4 vecinos de una celda de un color son siempre del color opuesto, entonces todas las celdas del mismo color se pueden actualizar en paralelo sin estorbarse entre si. Cada iteración pasa a tener 2 etapas, primero actualizar todas las rojas (con una barrera implícita) y luego actualizar todas las negras (otra barrera implícita).

Este cambio finalmente paraleliza el hotspot principal del programa. El resultado es el salto de rendimiento más grande del proyecto. En la configuración default ($n = 256$, $f = 6$), el FPS pasa de 25.20 (Paso 2) a 88.27 (Paso 3), un $3.5\times$ adicional.

7.4. Paso 4. Ajuste de schedule

Con el hotspot ya paralelo, se probó si el `schedule(dynamic, 16)` usado en todo el código era la mejor elección. Se cambiaron temporalmente los `#pragma` a `schedule(runtime)` (que se puede configurar sin recompilar via la variable de entorno `OMP_SCHEDULE`) y se barrió `static`, `dynamic` y `guided` con distintos tamaños de chunk, en malla chica y malla grande (Tabla 6).

El resultado principal: el `dynamic, 16` usado en todo el código es buena elección en malla chica, pero es la peor opción razonable en malla grande. `static` (con el chunk por defecto del compilador) le saca $2.5\times$ a `dynamic, 16` en $n = 700$ (54.47 vs. 21.93 FPS).

Con este resultado, se aplicó `schedule(static)` de verdad al código de la rama `04-schedule-tuning`, reemplazando el `schedule(dynamic, 16)` hardcodeado en todos los sitios del catálogo.

Al remedir la batería estándar con el cambio real (no solo con `OMP_SCHEDULE`), el resultado es un **tradeoff**, no una mejora universal. En malla chica `static` pierde un poco contra el `dynamic, 16` anterior. La fila B6 baja de 88.27 a 80.68 FPS, alrededor de un 8.6 % menos. En malla grande, en cambio, gana fuerte. La fila I6 sube de 28.32 a 53.47 FPS (casi el doble) y la fila C6 sube de 45.79 a 68.57 FPS. Ver Tabla 3 para los números completos.

7.5. Paso 6. Flags de compilacion

Se probo cambiar el flag de optimización del compilador de `-O2` (usado hasta ahora) a `-O3`, y adicionalmente `-march=native` para autovectorizacion especifica del CPU de la maquina de pruebas. Medido sobre el código actual (con `schedule(static)` ya aplicado, no el `dynamic, 16` viejo). Los resultados están en la Tabla 7.

Con `static` ya puesto, las 3 builds quedan dentro de un rango de pocos FPS entre sí en malla grande (la fila de `-O2` en la Tabla 7 usa la mejor de 3 corridas repetidas, 49.82, 51.22 y 50.28 FPS, medidas en distintos momentos). Ese rango de variación entre corridas del mismo build es casi tan ancho como la diferencia entre las 3 builds, así que no hay nada significativo que concluir entre `-O2`, `-O3` y `-march=native` a esta resolución de medición. Una medición anterior (hecha todavía con el `schedule(dynamic, 16)` viejo) había mostrado a `-O3` con una ganancia mayor en malla grande, pero dado el nivel de ruido visto acá, esa diferencia tampoco se puede tomar como concluyente sin promediar varias corridas.

8. Mecanismos de sincronización

- `#pragma omp parallel for`: la forma principal de paralelismo en el proyecto. Reparte las iteraciones de un loop `for` entre los hilos disponibles, con una barrera implícita al final (todos los hilos esperan a que termine el último antes de seguir).
- `#pragma omp parallel + #pragma omp for`: usado en el solver (Sección 7.3) para abrir una sola región paralela que envuelve las 20 iteraciones del Gauss-Seidel red-black, en vez de abrir/cerrar el team de hilos 40 veces por frame evitando el costo fijo de creación de hilos.
- `#pragma omp single`: dentro de la región paralela del solver, aplica la condición de frontera con un solo hilo mientras el resto espera en la barrera implícita del `single`, evitando que varios hilos escriban la misma celda de borde a la vez.
- `schedule(dynamic, chunk)`: política de reparto de trabajo por defecto usada en todo el código. Reparte bloques de `chunk` iteraciones bajo demanda: un hilo que termina su bloque pide el siguiente en vez de quedarse ocioso esperando a los demás. La Sección 7.4 muestra que no siempre es la mejor opción.
- `collapse(2)`: en los loops anidados de advección, proyección y render, funde las dos dimensiones (fila, columna) en un solo rango repartible. Sin esto, una malla más angosta que la cantidad de hilos dejaría núcleos sin trabajo.
- **Variables privadas** (`private(...)`): cada hilo necesita su propia copia de las variables escalares temporales del cuerpo del loop (por ejemplo, coordenadas de origen interpoladas en advección).

No se usaron mutex ni secciones críticas explícitas. El diseño evita condiciones de carrera por construcción (cada celda de salida se escribe una sola vez, por un solo hilo, en cada paso), así que las barreras implícitas de `omp for` y el `single` para la frontera son suficientes.

9. Resultados

9.1. Tabla maestra

La Tabla 3 muestra el FPS promedio por configuración (malla $\times$ fuentes), en cada paso de la cadena, con 20 hilos donde aplica. Metodología: cada fila se corrió con semilla fija (`-s 42`), se promedió el FPS ignorando las primeras líneas de calentamiento.

Fila	sequential	01-rb-tree	02-omp-loops	03-omp-solver (dynamic,16)	04-schedule (static)
A6 (n=64, f=6)	17.21	17.19	118.40	88.04	83.87
A12 (n=64, f=12)	17.16	17.20	117.43	88.31	84.39
B6 (n=256, f=6)	10.52	10.51	25.20	88.27	80.68
B12 (n=256, f=12)	10.95	10.95	25.24	88.66	80.92
C6 (n=512, f=6)	3.75	3.76	6.72	45.79	68.57
C12 (n=512, f=12)	4.30	4.30	6.89	52.33	69.53
H6 (n=600, f=6)	2.69	2.69	4.46	33.72	61.41
H12 (n=600, f=12)	3.09	3.09	4.79	41.39	62.29
I6 (n=700, f=6)	1.90	1.91	3.00	28.32	53.47
I12 (n=700, f=12)	2.10	2.10	3.21	35.57	54.76
D6 (n=1024, f=6)	0.92	0.92	1.15	5.10	4.64
D12 (n=1024, f=12)	1.00	0.99	1.23	6.32	3.21
E (n=256, f=4)	7.52	7.52	24.39	87.09	n/m
F (n=256, f=64)	11.39	11.39	25.36	87.28	n/m
G (n=256, f=256)	11.37	11.38	25.30	85.07	n/m

Cuadro 3: FPS promedio por paso de la cadena de paralelización. n/m = no medido con el schedule nuevo todavia.

En la configuración default ($n = 256$, $f = 6$), el FPS pasa de 10.52 (secuencial) a 88.27 (Paso 3), un speedup acumulado de $\sim 8,4\times$. El Paso 4 (`static`) es un tradeoff frente a Paso 3: pierde un poco en malla chica pero gana fuerte en malla grande (hasta $\sim +89\%$ en $n=700$, Tabla 3).

9.2. Escalabilidad por hilos

La Tabla 4 muestra el FPS variando `OMP_NUM_THREADS` en la fila B ($n=256$, $f=6$). Antes del Paso 3 (solver todavia secuencial), el escalado se estanca alrededor de $2,6\times$ (techo de Amdahl, el solver secuencial limita cuánto puede acelerar el resto). Desde el Paso 3 en adelante, con el solver ya paralelo, el escalado sube a $\sim 6\times$.

Hilos	FPS 02-omp-loops	FPS 03-omp-solver	FPS 04-schedule
1	9.95	14.72	14.72
2	14.94	25.39	25.52
4	20.33	45.94	45.91
8	23.57	67.84	68.27
16	25.52	83.27	83.66
20	25.57	87.76	87.72

Cuadro 4: FPS vs. cantidad de hilos, fila B ($n=256$, $f=6$).

También se corrió la misma curva en una malla grande ($n=700$, $f=12$), comparando el `schedule` actual contra `static` (el mejor encontrado para malla grande, Tabla 5). Con `static`, 20 hilos da 48.11 FPS contra los 34.59 FPS del `dynamic`, 16 actual en la misma fila: +39 % solo cambiando el `schedule`.

Hilos	FPS <code>static</code>	FPS <code>dynamic</code> , 16 (actual)
1	6.82	—
2	15.25	—
4	30.58	—
8	41.02	—
16	53.80	—
20	48.11	34.59

Cuadro 5: FPS vs. hilos en malla grande ($n=700$, $f=12$).

Con `static` el pico está en 16 hilos, no en 20: a 700 filas, el reparto estatico en 20 partes no cae tan parejo como en 16.

9.3. Barrido de `schedule/chunk`

OMP_SCHEDULE	FPS $n=256$	FPS $n=700$
<code>static</code> (default, actual)	83.40	54.47
<code>static</code> , 4	83.57	43.01
<code>static</code> , 16	84.53	46.87
<code>static</code> , 64	72.89	44.42
<code>dynamic</code> , 1	31.19	5.55
<code>dynamic</code> , 4	64.36	15.77
<code>dynamic</code> , 16 (viejo default)	87.58	21.93
<code>dynamic</code> , 64	76.88	2.02
<code>guided</code> (default)	96.80	49.42
<code>guided</code> , 16	94.71	3.43

Cuadro 6: Barrido de `schedule/chunk`, 20 hilos, $f=6$.

9.4. Flags de build

Build	FPS $n=256, f=6$	FPS $n=700, f=6$
-O2 (actual)	80.18	51.22
-O3	78.94	46.41
-O3 -march=native	75.52	42.72

Cuadro 7: FPS por flag de optimización, 20 hilos, sobre el código actual (`schedule(static)` ya aplicado).

10. Conclusiones

- Paralelizar el solver de presión/difusión (red-black Gauss-Seidel) fue, por lejos, el cambio de mayor impacto. El FPS default subió de 25.20 a 88.27 ($3,5\times$), y la escalabilidad por hilos paso de $\sim 2,6\times$ a $\sim 6\times$.
- Antes de paralelizar el hotspot real, paralelizar el resto del programa (Paso 2) ayuda, pero el techo de Amdahl limita cuanto se puede ganar mientras el cuello de botella siga secuencial.
- La elección de `schedule` de OpenMP importa más de lo esperado y no tiene un ganador universal: `dynamic`, 16 era mejor en malla chica, `static` gana fuerte en malla grande ($2,5\times$ en el barrido, hasta $+89\%$ al aplicarlo de verdad al código). Se aplicó `static` a la rama `04-schedule-tuning`; el resultado real confirma el tradeoff, pierde $\sim 8\%$ en malla chica para ganar hasta $\sim 89\%$ en malla grande.
- Con `static` ya aplicado, la diferencia entre `-O2`, `-O3` y `-march=native` quedó dentro del ruido de medición de una sola corrida (repetir `-O2` varias veces movió el resultado casi tanto como el que supuesto cambio de build). No hay evidencia concluyente de que alguna build sea mejor a esta resolución de medición.
- Barnes-Hut para el n-body no se nota en el FPS a las resoluciones probadas porque el solver de fluidos domina el costo total; el algoritmo de n-body es una fracción pequeña del trabajo por frame.

11. Recomendaciones

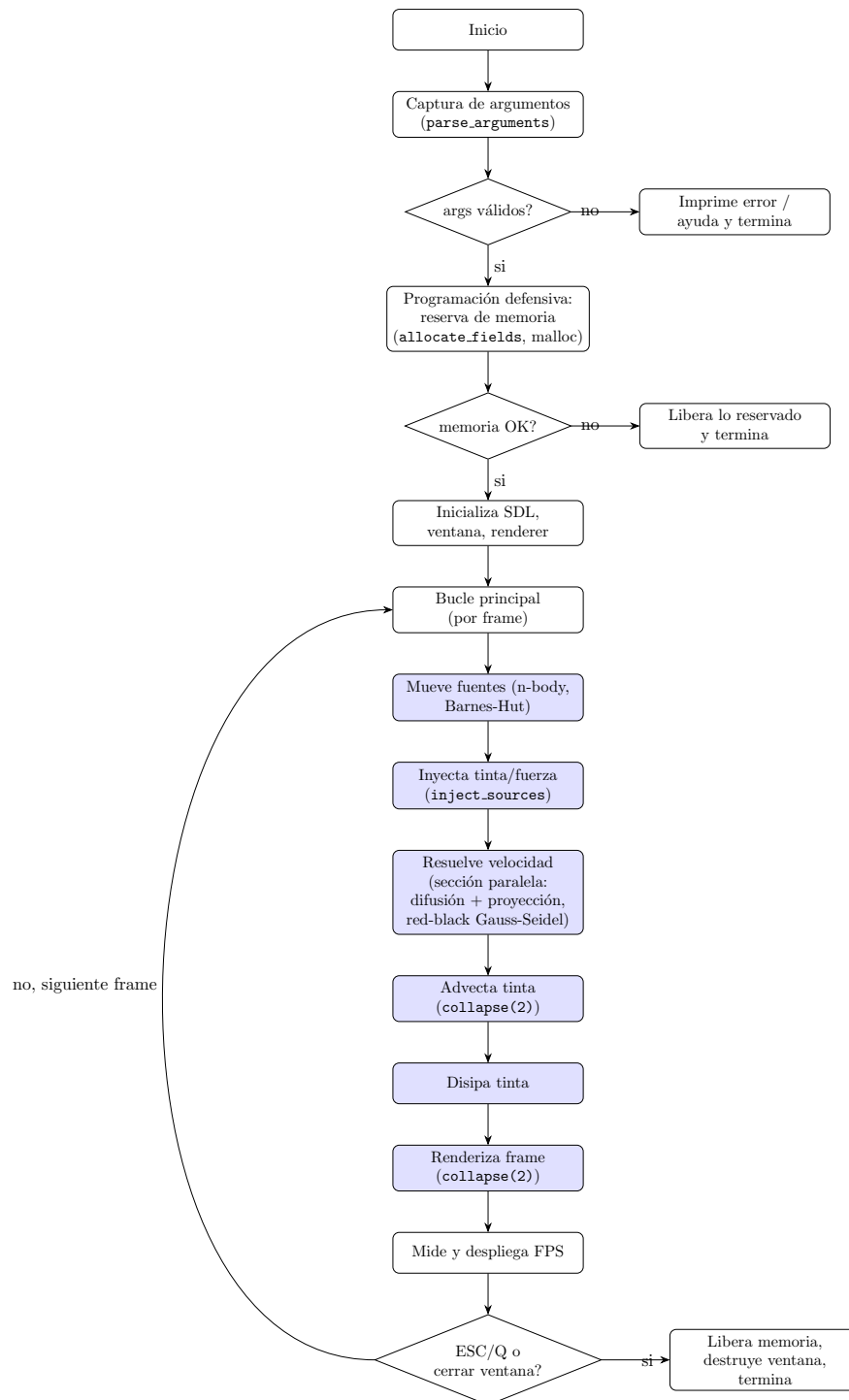
- Ya se aplicó `static` en `04-schedule-tuning`. Si el uso real del programa favorece mallas chicas ($n \leq 256$), vale la pena reconsiderar, ya que ahí `dynamic`, 16 sigue siendo mejor; para mallas grandes, `static` es la mejor elección medida. Una opción futura es dejarlo configurable via `schedule(runtime)` y elegir en base a `-n` al arrancar.
- Cambiar `-O2` a `-O3` en el `Makefile` no mostró una ganancia distinguible del ruido de una sola corrida. Si se quiere una respuesta real, hay que promediar varias corridas por build antes de decidir, no una sola medición.
- Retomar el Paso 5 (`collapse(2)`) con una metodología más controlada (por ejemplo, sin renderizado en pantalla completa) si se quiere ese número para mallas chicas con pocos núcleos.
- Para mallas mayores a $n = 1024$ o cantidades de fuentes mayores a 256, medir de nuevo, ya que el comportamiento del schedule óptimo podría cambiar fuera del rango probado.

Referencias

- [1] Stam, J. (1999). *Stable Fluids*. En Proceedings of the 26th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '99), 121-128.
- [2] Barnes, J., & Hut, P. (1986). *A hierarchical $O(N \log N)$ force-calculation algorithm*. Nature, 324(6096), 446-449.
- [3] OpenMP Architecture Review Board. (2021). *OpenMP Application Programming Interface, Versión 5.2*. <https://www.openmp.org/specifications/>

-
- [4] Chapman, B., Jost, G., & Van Der Pas, R. (2007). *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press.

A. Anexo 1. Diagrama de flujo del programa



Los bloques en azul son las secciones paralelizadas con OpenMP. La sincronización entre ellas es implícita: cada `#pragma omp for` tiene una barrera al final, y la sección de velocidad usa `#pragma omp single` para la condición de frontera dentro de su propia región paralela (ver Sección 7.3).

Que pasa dentro de cada bloque, un frame a la vez

El diagrama de arriba muestra el orden, pero no el detalle de cada bloque. Esta sección entra a ese detalle, bloque por bloque, en el orden real de ejecución dentro de `main.c`.

1. Mueve fuentes (n-body, Barnes-Hut, `update_nbody_sources`). Reconstruye un quadtree desde cero cada frame (las fuentes se mueven, no tiene sentido mantenerlo entre frames). Primero inserta las f fuentes una por una (`bh_insert`, recursivo, subdivide un nodo hoja en 4 cuando cae una segunda fuente adentro). Después, para cada fuente, recorre el árbol (`bh_accumulate`), y si un nodo está lo bastante lejos en relación a su tamaño ($\theta = 0,5$), se trata como un solo punto de masa en vez de bajar a sus 4 hijos, y eso es lo que baja el costo de $O(f^2)$ a $O(f \log f)$. Con la aceleración acumulada, integra velocidad y posición, y aplica un rebote amortiguado si una fuente llegaría a salir del área válida de inyección. Esta etapa sigue secuencial en toda la cadena. El árbol se construye una vez por frame y depende del resultado del frame anterior en cada inserción, no vale la pena paralelizarla a la cantidad de fuentes que se usan típicamente ($f \leq 256$).

2. Inyecta tinta/fuerza (`inject_sources`). Primero limpia los buffers “previous” de velocidad y tinta (van a recibirse de nuevo este frame). Para cada fuente, calcula dos gaussianas centradas en su posición (continua, no la celda entera), una ancha que da el color/empuje general, y una angosta (“core”) que agrega un núcleo brillante casi blanco en el centro. La dirección del chorro rota con $\cos(\text{phase})/\sin(\text{phase})$, generando el efecto espiral. No está paralelizada, ya que el radio de inyección por fuente es chico (unas pocas celdas) y varias fuentes pueden solaparse en la misma celda si están cerca, lo que forzaría una sección crítica o reducción, algo que no vale la pena al tamaño típico de $-f$.

3. Resuelve velocidad (`velocity_step`). El bloque más caro del frame, y donde vive el paralelismo real. Es una secuencia fija, primero fuerza $\rightarrow$ difusión $\rightarrow$ proyección $\rightarrow$ autoadvección $\rightarrow$ proyección otra vez (difundir y autoadveccionar cada uno reintroduce algo de divergencia, por eso se proyecta dos veces). La difusión y cada proyección llaman a `solve_linear`, que abre *una sola* región `#pragma omp parallel` y adentro corre las 20 iteraciones de Gauss-Seidel red-black, 20 veces (actualizar celdas rojas, barrera implícita, actualizar celdas negras, barrera implícita). Abrir la región paralela una sola vez en vez de 40 veces (20 iteraciones $\times$ 2 colores) evita el costo fijo de crear el team de hilos repetidamente, que en mallas chicas se nota. La condición de frontera se aplica con `#pragma omp single` dentro de esa misma región: un solo hilo la escribe mientras los demás esperan en la barrera implícita del `single`, sin salir de la región paralela.

4. Advecciona tinta (`ink_step`, una vez por canal R, G, B). Advección semi-Lagrangiana. Para cada celda destino, calcula de donde vino ese fluido dando un paso hacia atrás a lo largo de su propio campo de velocidad, y agarra el valor de ahí por interpolación bilineal de las 4 celdas reales alrededor de ese punto (fraccional). Cada celda destino solo lee el campo anterior, nunca lo escribe, así que es paralelizable sin ningún cuidado extra. Usa `#pragma omp parallel for collapse(2)`: funde el loop de i y el de j en un solo rango repartible, para que una malla angosta (menos filas que hilos) siga repartiendo trabajo entre todos. Se llama 3 veces (una por canal de color), cada una independiente de las otras.

5. Disipa tinta (`dissipate_ink`). Multiplica cada celda de cada canal por `DISSIPATION` (0.995), sin eso la tinta se acumularía hasta saturar la pantalla a blanco. Loop plano sobre `celdas_totales`, cada celda independiente, `#pragma omp parallel for`.

6. Renderiza frame (`render_ink`). Recorre cada pixel de la textura (no cada celda de la malla, ya que la resolución de ventana suele ser mayor que `-n`), interpola bilinealmente la densidad de tinta en esa posición, aplica un mapeo de tono estilo Reinhard y un ajuste de saturación, y calcula el canal alfa a partir del brillo máximo de los 3 canales (para que el fondo se vea donde no hay tinta). También usa `collapse(2)`: sin él, con una ventana más angosta que alta (o viceversa) los hilos no se repartirían parejo solo sobre el loop externo.

7. Mide FPS y actualiza pantalla. Cuenta frames acumulados y, cada ~ 0.5 s de tiempo real, calcula el promedio, lo imprime a `stdout` y lo pone en el título de la ventana. No es parte del cálculo de la simulación, es solo instrumentación.

B. Anexo 2. Catálogo de funciones

config.c

Función	Entradas	Salidas	Descripción
<code>parse_arguments</code>	<code>argc</code> (int), <code>argv</code> (char**)	<code>config</code> (Config*, por referencia), retorna int (1 ok, 0 error, -1 ayuda)	Recorre <code>argv</code> y llena <code>config</code> con los valores validados.
<code>read_int</code>	<code>text</code> (char*), rango min/max	<code>out</code> (int*), retorna 1/0	Convierte texto a int validando formato y rango.
<code>read_float</code>	<code>text</code> (char*), rango min/max	<code>out</code> (float*), retorna 1/0	Igual que <code>read_int</code> pero para float.
<code>print_help</code>	<code>program_name</code> (char*)	ninguna (imprime a stdout)	Imprime la pantalla de uso del programa.

fields.c

Función	Entradas	Salidas	Descripción
<code>allocate_fields</code>	<code>fields</code> (FluidFields*), <code>resolution</code> (int)	retorna int (1 ok, 0 error)	Reserva memoria para los 12 campos de la simulación (velocidad, tinta, presión, divergencia).
<code>free_fields</code>	<code>fields</code> (FluidFields*)	ninguna	Libera los 12 campos y pone los punteros en NULL.
<code>clear_fields</code>	<code>fields</code> (FluidFields*)	ninguna	Pone todos los campos en cero (usado al reiniciar con la tecla R).

sources.c

Función	Entradas	Salidas	Descripción
<code>init_sources</code>	<code>sources</code> (InkSource*), <code>count</code> , <code>resolution</code> (int)	ninguna	Inicializa posición, velocidad, masa, color y fuerza de cada fuente.

<code>inject_sources</code>	<code>sources</code> (InkSource*), <code>count</code> (int), <code>fields</code> (FluidFields*), <code>aspect</code> (float)	ninguna (modifica <code>fields</code>)	Deposita tinta y momentum de cada fuente en un vecindario gaussiano.
<code>dissipate_ink</code>	<code>fields</code> (FluidFields*)	ninguna	Reduce la tinta de cada celda por un factor fijo (paralelo con <code>omp parallel for</code>).

`solver.c`

Función	Entradas	Salidas	Descripción
<code>apply_boundary</code>	<code>bnd_type</code> (int), <code>field</code> (float*)	ninguna	Aplica condiciones de frontera (rebote en paredes).
<code>add_source</code>	<code>dest</code> , <code>source</code> (float*), <code>dt</code> (float), <code>n</code> (int)	ninguna	Suma el término fuente a cada celda (paralelo).
<code>relax_color</code>	<code>field</code> , <code>field_prev</code> (float*), <code>a</code> , <code>inv_c</code> (float), <code>parity</code> (int)	ninguna	Actualiza las celdas de una paridad del tablero red-black (paralelo, dentro de región <code>omp parallel</code>).
<code>solve_linear</code>	<code>bnd_type</code> (int), <code>field</code> , <code>field_prev</code> (float*), <code>a</code> , <code>c</code> (float)	ninguna	Corre las 20 iteraciones de Gauss-Seidel red-black dentro de una sola región paralela.
<code>diffuse</code>	<code>bnd_type</code> (int), <code>field</code> , <code>field_prev</code> (float*), <code>diff</code> , <code>dt</code> (float), <code>n</code> (int)	ninguna	Arma los coeficientes y llama a <code>solve_linear</code> para difusión implícita.
<code>advect</code>	<code>bnd_type</code> (int), <code>field</code> , <code>field_prev</code> (float*), <code>vel_x</code> , <code>vel_y</code> (float*), <code>dt</code> (float)	ninguna	Advección semi-Lagrangiana (paralelo, <code>collapse(2)</code>).
<code>project</code>	<code>vel_x</code> , <code>vel_y</code> , <code>pressure</code> , <code>divergence</code> (float*)	ninguna	Proyección de Hodge: mide divergencia, resuelve presión, resta el gradiente de la velocidad.
<code>swap_fields</code>	<code>field_a</code> , <code>field_b</code> (float**)	ninguna (intercambia punteros)	Intercambia los punteros de dos buffers de campo.

<code>ink_step</code>	<code>ink, ink_prev (float**), vel_x, vel_y (float*), diff, dt (float), n (int)</code>	ninguna	Un paso completo (difusión + advección) para un canal de tinta.
<code>velocity_step</code>	<code>fields (FluidFields*), viscosity, dt (float)</code>	ninguna	Un paso completo de velocidad: fuerza, difusión, proyección, autoadveccion, proyección.

`nbody.c`

Función	Entradas	Salidas	Descripción
<code>bh_new_node</code>	<code>cx, cy, half (float)</code>	retorna int (índice del nodo)	Crea un nodo nuevo del quadtree Barnes-Hut.
<code>bh_quadrant</code>	<code>n (QuadNode*), x, y (float)</code>	retorna int (0-3)	Determina en que cuadrante cae un punto.
<code>bh_subdivide</code>	<code>idx (int)</code>	ninguna	Crea los 4 hijos de un nodo.
<code>bh_insert</code>	<code>idx, body (int), sources (InkSource*), depth (int)</code>	ninguna	Inserta una fuente en el árbol recursivamente.
<code>bh_accumulate</code>	<code>idx, body (int), sources (InkSource*), acc_x, acc_y (float*)</code>	ninguna (acumula en <code>acc_x</code> / <code>acc_y</code>)	Recorre el árbol acumulando aceleración gravitacional sobre una fuente.
<code>update_nbody_sources</code>	<code>sources (InkSource*), count, resolution (int)</code>	ninguna	Reconstruye el árbol, calcula gravedad, integra posición/velocidad y aplica rebote en bordes.

`render.c`

Función	Entradas	Salidas	Descripción
<code>sample_bilinear</code>	<code>field (float*), fx, fy (float)</code>	retorna float	Interpola bilinealmente un campo en una posición continua.

<code>render_ink</code>	<code>texture</code> (<code>SDL_Texture*</code>), <code>fields</code> (<code>FluidFields*</code>), <code>width</code> , <code>height</code> (int)	ninguna	Vuelca la densidad de tinta a la textura SDL, un pixel a la vez (paralelo, <code>collapse(2)</code>).
-------------------------	--	---------	--

`background.c`

Función	Entradas	Salidas	Descripción
<code>init_background</code>	<code>stars</code> (<code>Star*</code>), <code>count</code> , <code>width</code> , <code>height</code> (int)	ninguna	Coloca estrellas de fondo en posiciones aleatorias.
<code>update_background</code>	<code>stars</code> (<code>Star*</code>), <code>count</code> (int)	ninguna	Avanza el parpadeo de cada estrella.
<code>draw_background</code>	<code>renderer</code> (<code>SDL_Renderer*</code>), <code>stars</code> (<code>Star*</code>), <code>count</code> (int)	ninguna	Dibuja las estrellas de fondo.

`utils.c`

Función	Entradas	Salidas	Descripción
<code>random_range</code>	<code>min</code> , <code>max</code> (float)	retorna float	Número pseudoaleatorio uniforme en $[min, max]$.
<code>clamp</code>	<code>value</code> , <code>min</code> , <code>max</code> (float)	retorna float	Limita un valor al rango $[min, max]$.

`main.c`

Función	Entradas	Salidas	Descripción
<code>main</code>	<code>argc</code> (int), <code>argv</code> (<code>char**</code>)	retorna int (código de salida)	Punto de entrada: parsea argumentos, inicializa SDL y memoria, corre el bucle principal, libera todo al salir.

C. Anexo 3. Bitácora de pruebas

Metodología

Cada configuración se corrió con semilla fija (`-s 42`), ignorando las primeras líneas de FPS (calentamiento/cache fría), y promediando el resto. La batería estándar (15 configuraciones, Tabla 17) se repitió idéntica después de cada paso de la cadena; son 15 mediciones por paso, más de las 10 mínimas requeridas.

Fila	-n	-f	Motivo
A6/A12	64	6/12	mallla chica, referencia rápida
B6/B12	256	6/12	configuración default
C6/C12	512	6/12	mallla grande, domina el solver
H6/H12	600	6/12	rango de FPS medio observado
I6/I12	700	6/12	extremo superior de ese rango
D6/D12	1024	6/12	mallla muy grande, límite superior
E	256	4	pocas fuentes
F	256	64	muchas fuentes
G	256	256	límite superior de fuentes

Cuadro 17: Batería estándar de pruebas (15 configuraciones).

Para el barrido de `schedule` (Tabla 6) se probaron 10 combinaciones de política/chunk, cada una en 2 tamaños de mallla, 20 mediciones en total. Para la escalabilidad por hilos (Tablas 4 y 5) se probaron 6 cantidades de hilos por curva.

Captura de una corrida

A continuación, salida real de consola de una corrida de referencia ($n = 256$, $f = 6$, semilla 42), mostrando el arranque, calentamiento y estabilización del FPS.

```
Fluid simulation (Navier-Stokes) - SEQUENTIAL version
Grid       : 256 x 256 cells (65536 interior cells)
Sources    : 6
Window     : 1920 x 1080 px
Seed       : 42
visc / diff : 0.00000 / 0.00010
FPS= 12.73
FPS= 11.41
FPS= 12.96
FPS= 13.80
FPS= 75.53
FPS= 83.46
FPS= 83.82
FPS= 82.31
FPS= 84.26
FPS= 85.41
FPS= 84.09
FPS= 63.32
FPS= 74.58
FPS= 70.27
FPS= 77.99
```

La etiqueta "SEQUENTIAL version.^{en} el encabezado es texto residual del archivo `main.c` (no se actualizó al paralelizar); el binario usado en esta corrida es la versión paralela con OpenMP habilitado (20 hilos).

Comando usado

```
1 timeout 30s ./ss -n <N> -f <F> -s 42 2>/dev/null \  
2 | grep "FPS=" | tail -n +6 \  
3 | awk -F'= ' '{s+=$2; c++} END {printf "FPS promedio: %.2f\n", s/c}'
```

El resto de las mediciones completas (las 15 filas de la batería estándar por cada paso, el barrido de schedule y las curvas de escalabilidad) están en las tablas de la Sección 9 de este mismo informe. El historial de commits del repositorio muestra cuándo se corrió cada batería.